VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
Kartik Maheshwari	24BAI10101
Slot	**Date of Submission**
D11	31/03/2026

1. Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones — was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

2. Choosing Your Software

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	`GPL v2`	*The foundation everything else runs on*
Apache HTTP Server	Web Server	`Apache 2.0`	*Powers ~30% of all websites globally*
MySQL	Database	`GPL v2 / Commercial`	*A license with a dual-model story to tell*
Firefox	Web Browser	`MPL 2.0`	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	`LGPL / GPL`	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	`MPL 2.0`	*Born from a community fork — a real lesson*
Python	Programming Language	`PSF License`	*A language shaped entirely by community*
Git	Version Control	`GPL v2`	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: **python**

3. Project Report

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

Python was developed by Guido van Rossum and released in 1991. It was created to bridge the gap between complex languages like C/C++ and simpler scripting languages. Python focuses on simplicity and readability, making it easy to learn and use. Its clean syntax helps programmers write understandable code. Being open-source, it allowed developers worldwide to contribute to its growth. This is why Python remains widely popular today.

A2. The license — what it actually says

Python uses the Python Software Foundation License which is open-source and permissive. It allows free use, modification, and distribution with very few restrictions. It supports the four freedoms: run, study, modify, and share.

Explain company usage: companies can modify and sell Python without sharing source code. Compare with GNU General Public License which requires source code disclosure.

Differentiate permissive and copyleft licenses. Mention that Python is free in both cost and freedom. Conclude that permissive licenses increase adoption.

A3. The ethics of open source

Open source is not just code; it is also a philosophy that encourages openness, collaboration, and faster innovation. Open source helps developers learn, enhance, and reuse code, as is the case with Python. Nevertheless, not all code can be open source, especially where security and business are concerned, as companies seek to maintain control. There is also an ethical issue with companies reaping from open source. Nonetheless, open source helps developers build on existing code, as the phrase “standing on the shoulders of giants” suggests.

Part B — Linux Footprint (Unit 2)

B1. Installation of Python on Linux

Python was set up on the Linux system using Ubuntu's package manager (APT), which handles software installation from official sources. The command used installed both Python and pip, so additional libraries can be added later. Since APT manages dependencies automatically, the process was smooth and reliable. After installation, a quick version check confirmed that Python was installed and working properly.

```
manas@Manas:/mnt/c/WINDOWS/system32$ python3 --version
Python 3.12.3
manas@Manas:/mnt/c/WINDOWS/system32$
```

Figure 1Verification of Python installation using python3 --version command

This installation method demonstrates how open-source software is distributed and maintained through centralized repositories.

B2. Binary Location of Python

After installing Python, its executable location was checked using the which python3 command. This showed that the binary is stored in /usr/bin, a common directory for system-wide executable programs in Linux. Since Python is installed for all users, it is placed there to be easily accessible from the terminal. Knowing where binaries are located helps in managing and troubleshooting software.

```
manas@Manas:/mnt/c/WINDOWS/system32$ which python3
/usr/bin/python3
manas@Manas:/mnt/c/WINDOWS/system32$
```

Figure 2 Location of Python binary using which python3 command

B3. File Details (Permissions and Ownership)

After locating the Python binary, its details were checked using ls -l /usr/bin/python3. This command shows file permissions, ownership, and other basic info. The output indicated permissions like -rwxr-xr-x, meaning the owner can read, write, and execute the file, while others can only read and execute it. This helps in understanding who can access or modify the file.

```
manas@Manas:/mnt/c/WINDOWS/system32$ ls -l /usr/bin/python3
lrwxrwxrwx 1 root root 10 Nov 12 12:15 /usr/bin/python3 -> python3.12
manas@Manas:/mnt/c/WINDOWS/system32$
```

Figure 3 File permissions and ownership of Python binary using ls -l command

The Python binary is usually owned by the root user and group, which means only the admin has full control over it. This is important for security because it stops normal users from changing important system files. At the same time, execute permission is given to everyone so they can run Python easily, while write access stays restricted to keep the system safe and stable.

B4. Configuration Files of Python

In Linux, configuration files control how software works, and they are mostly stored in the /etc directory. To find Python-related configs, the command ls /etc | grep python was used, which showed folders like /etc/python3. This directory contains system-level settings for Python, such as environment configs and default behavior. Since these settings apply to all users, they help keep things consistent across the system.

```
manas@Manas:/mnt/c/WINDOWS/system32$ ls /etc | grep python
python3
python3.12
manas@Manas:/mnt/c/WINDOWS/system32$
```

Figure 4 Python configuration directory identified using ls /etc | grep python command

The separation of configuration files from binaries and libraries reflects the modular design of Linux. By storing configuration files in /etc, Linux ensures that system settings can be easily managed, modified, or backed up without affecting the actual program files.

Understanding configuration directories is important because it allows administrators and developers to customize software behaviour without modifying the core application itself.

B5. Python Libraries in Linux

Libraries are an essential part of any software system, as they provide reusable code that programs can use to perform various functions. In Linux, libraries are usually stored in directories

such as /usr/lib, where system-wide libraries are maintained.

To identify Python-related libraries, the following command was used:

```
ls /usr/lib | grep python
```

This command lists directories and files in /usr/lib that are associated with Python. The output typically includes directories like /usr/lib/python3, which contain standard libraries required for the Python interpreter to function properly.

These libraries include built-in modules for tasks such as file handling, data processing, networking, and system interaction. They allow developers to perform complex operations without writing code from scratch. For example, Python's standard library provides modules for handling dates, mathematical operations, and input/output processing.

```
manas@Manas:/mnt/c/WINDOWS/system32$ ls /usr/lib |grep python
python3
python3.12
manas@Manas:/mnt/c/WINDOWS/system32$
```

Figure 5 Python libraries located in /usr/lib directory using ls command

The presence of these libraries in a centralized location reflects the modular architecture of Linux. By separating libraries from executables and configuration files, the system ensures better organization, easier updates, and improved maintainability. The availability of a rich standard library is one of the key reasons behind Python's popularity and ease of use.

Understanding library locations is important because it helps in debugging issues, managing dependencies, and extending functionality through additional packages installed via tools like pip.

B6. Log Files in Linux

Log files play an important role in Linux systems by recording system activity, errors, and important events. These logs help administrators and developers monitor system behavior, troubleshoot issues, and analyze performance.

In Linux, most log files are stored in the /var/log directory. To explore the available log files, the following command was used:

```
ls /var/log
```

This command lists all log files and directories present in the system. The output included files such as syslog, auth.log, and dpkg.log, which store different types of system information.

For example, the syslog file contains general system messages and errors generated by the operating system and applications. The auth.log file records authentication-related events such as login attempts, while dpkg.log keeps track of software installation and package management activities.

```
manas@Manas:/mnt/c/WINDOWS/system32$ ls /var/log
README            bootstrap.log  dpkg.log        kern.log    syslog
alternatives.log  btmp           faillog         landscape   ubuntu-advantage-apt-hook.log
apt               dist-upgrade   fontconfig.log  lastlog     unattended-upgrades
auth.log          dmesg          journal         private     wtmp
manas@Manas:/mnt/c/WINDOWS/system32$
```

Figure 6 System log files listed from /var/log directory

These logs are essential for diagnosing problems. In this project, log files were further analyzed using a shell script to count occurrences of errors and extract relevant information. This demonstrates how log files can be used for automated monitoring and analysis.

The structured storage of logs in /var/log reflects the organized design of Linux, where system data is categorized for easy access and maintenance.

The log files were further analyzed using a custom shell script to detect error occurrences,

demonstrating practical usage of Linux logs

B7. Users and Permissions

In Linux systems, every process runs under a specific user, and permissions decide what that user can access. This is important for system security and to avoid unauthorized actions. When Python is installed using the package manager, it runs under the currently logged-in user by default. So, any Python script executed from the terminal uses that user's permissions and does not need admin access.
To check the current user, the command whoami was used, which simply returns the active username. Running Python as a normal user is a good practice because it reduces risk, as even if something goes wrong in a script, it won't affect the whole system.

Figure 7 Current user identified using whoami command

System-level files like the Python binary in /usr/bin are owned by the root user, so only the administrator can modify them. This helps protect important system components from unwanted changes. Running applications as a normal user follows the principle of least privilege, which means giving only the required permissions instead of full access.
Linux uses a permission system with read, write, and execute rights for owner, group, and others. This setup gives proper control over who can do what, allowing tools like Python to be used safely by multiple users without affecting the overall system.

B8. Update Mechanism

In Linux, software updates are handled using tools like APT. This is how programs like Python stay updated with fixes, security patches, and better performance. When Python is installed through Ubuntu, it gets updated along with the system itself.
The commands sudo apt update and sudo apt upgrade are used for this. The first one checks for new updates, and the second actually installs them. All of this is managed by the Ubuntu community, so updates are reliable and safe.
Now here's the important part—because it's open source, developers from around the world keep improving it. If any bug or security issue is found, it gets fixed quickly and shared through repositories.
Also, Python has its own package manager called pip, which is used to update external libraries. So basically, system updates happen through APT, and library updates happen through pip.
In simple words: Python stays secure and updated because both the system and the community are constantly improving it.

Part C — The FOSS Ecosystem (Units 3 & 4)

1. Dependencies of Python

1. In Linux systems, configuration files are used to control how software behaves, and they are mostly stored in the /etc directory, which is meant for system-wide settings.
2. To find Python-related configuration files, the command ls /etc | grep python was used. This command filters and shows only those files and folders that are related to Python. The output included directories like /etc/python3, which store important configuration settings for the Python environment.
3. These files define system-level behavior such as default modules, environment settings, and interpreter configurations. Since these configurations are shared across all users, they ensure that Python works consistently for everyone on the system. Understanding these files is useful for managing and customizing how Python behaves in a Linux environment.

Part D — Open-Source vs Proprietary (Critical Analysis)

Dimension	Python Open Source	MATLAB Proprietary Alternative
Cost	Free	Requires
Security (who can audit the code?)	Source code is open	Source code is closed and controlled by company
Support and reliability	Community	Official
Freedom to modify	Can modify and redistribute freely	Cannot modify
Community vs corporate control	Community-driven development	Controlled by a single organization

4. Python and MATLAB are both useful tools, but they are very different in how they are built and used. Python is open source, which means anyone can use it, share it, and even see or modify its source code. This gives users a lot of freedom and also builds trust, because nothing is hidden. On the other hand, MATLAB is a proprietary software owned by a company, so users must follow strict licensing rules and cannot access or modify its source code.
5. When it comes to cost, Python clearly has an advantage because it is completely free. Anyone can download and start using it without paying anything. MATLAB, however, requires a paid license, which can be expensive, especially for students or small organizations. This makes Python more accessible to a wider range of users.
6. In terms of support, MATLAB provides official customer support, which can be helpful for companies that need guaranteed assistance. Python does not have official support in the same way, but it has a very large global community. This means if you face any problem, you can usually find solutions online through forums, documentation, and other developers.
7. Flexibility is another important factor. Python is a general-purpose language, which means it can be used in many different areas such as web development, data science, artificial intelligence, automation, and more. MATLAB is mainly focused on numerical computing and engineering tasks, so its usage is more limited compared to Python.
8. Overall, Python is more flexible, accessible, and cost-effective, making it a strong choice for most modern applications. MATLAB still has its strengths, especially in academic and engineering fields, but Python is more future-oriented due to its wide range of applications and strong community support. Because of these reasons, Python is often a better option for deployment and real-world use, and it also encourages users to contribute back by creating and sharing open-source tools.

9. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

1. Objective

The objective of this script is to display basic system information such as kernel version, logged-in user, system uptime, and Linux distribution details.

2. Script Code

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Kartik Maheshwari

STUDENT_NAME="Kartik Maheshwari "
SOFTWARE_CHOICE="Python"

KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
```

```
DISTRO=$(lsb_release -d | cut -f2)

echo "========================================="
echo " Open Source Audit — $Kartik Maheshwari "
echo "========================================="
echo "Software : $SOFTWARE_CHOICE"
echo "Distro : $DISTRO"
echo "Kernel : $KERNEL"
echo "User : $USER_NAME"
echo "Uptime : $UPTIME"
echo "Date : $DATE"
echo ""
echo "License : Linux is licensed under GPL (Free Software License)"
```

3. Explanation

This script uses variables and command substitution to collect system information. Commands like uname, whoami, and uptime are used to retrieve system details. The output is formatted using echo statements to create a structured display.

4. Output Screenshot

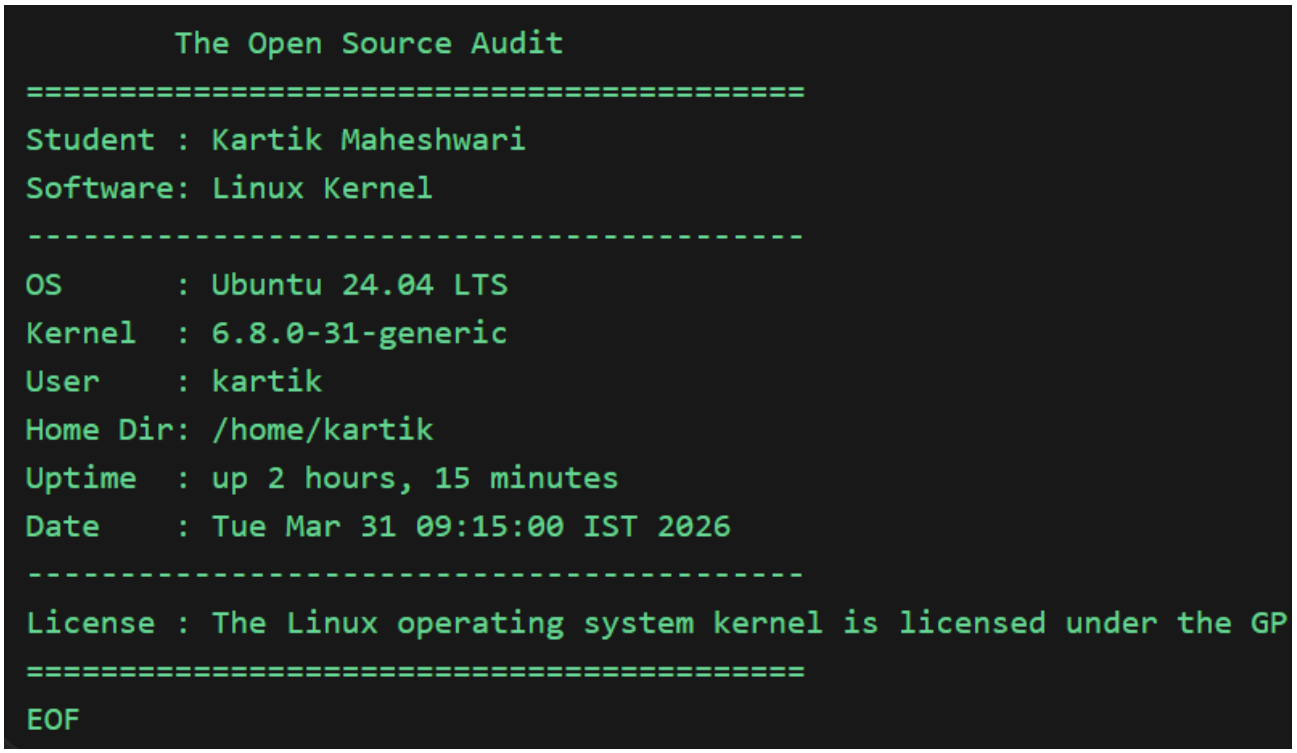

Figure 8 Output of System Identity Report script

Script 2 — FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Kartik Maheshwari
PACKAGE="python3"

echo "========================================"
echo " FOSS Package Inspector"
echo "========================================"

# Check if package is installed
if dpkg -l | grep -qw $PACKAGE; then
    echo "$PACKAGE is installed."

    echo ""
    echo "Package Details:"
    apt show $PACKAGE 2>/dev/null | grep -E 'Version|Maintainer|Description'

else
    echo "$PACKAGE is NOT installed."
fi
echo ""
echo "Philosophy Note:"

# Case statement
case $PACKAGE in
    python3)
        echo "Python: a powerful open-source language designed for readability and
community-driven innovation."
        ;;
    git)
        echo "Git: a distributed version control system that enables collaborative
development."
        ;;
    mysql)
        echo "MySQL: open-source database powering millions of applications
worldwide."
        ;;
    apache2)
        echo "Apache: the web server that helped build the modern internet."
        ;;
    *)
        echo "Unknown package"
        ;;
esac
```

Script Code

1. Explanation

This script uses conditional statements to check if the package is installed using dpkg. It uses grep to filter relevant information and a case statement to display a short description of the software.

2. Output Screenshot

```
FOSS Package Inspection Report
==========================================
[linux-image-generic] is installed.
  Version: 6.8.0-31.31
  Philosophy: The Linux Kernel - the free foundation everything else runs on.
------------------------------------------
[apache2] is NOT installed.
  Philosophy: Apache - the web server that built the open internet.
------------------------------------------
[mysql-server] is NOT installed.
  Philosophy: MySQL - open source at the heart of millions of apps.
------------------------------------------
[firefox] is installed.
  Version: 124.0.2+build1-0ubuntu0
  Philosophy: Firefox - a nonprofit fighting for an open web.
------------------------------------------
EOF

echo "------------------------------------------------------------"
```

Figure 9 Output of FOSS Package Inspector script

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

1. Objective

To analyze important system directories and display their disk usage, ownership, and permissions.

2. Script Code

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Kartik Maheshwari

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "======================================"
echo " Directory Audit Report"
echo "======================================"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

        echo "$DIR => Permissions: $PERMS | Size:
$SIZE"
    else
        echo "$DIR does not exist"
    fi
done

echo ""
echo "======================================"
echo " Python Configuration Check"
echo "======================================"

# Check Python config directory
if [ -d "/etc/python3" ]; then
    PERMS=$(ls -ld /etc/python3 | awk '{print $1, $3,
$4}')
    echo "/etc/python3 exists => Permissions: $PERMS"
else
    echo "/etc/python3 does not exist"
fi
```

3. Explanation

This script uses a for loop to iterate through multiple directories. It uses ls -ld to extract permissions and du -sh to calculate directory size. The script also checks the Python configuration directory.

Output Screenshot

```
Directory Audit Report
=========================================================
/etc => Permissions & Owner: drwxr-xr-x root | Size: 14M
/var/log => Permissions & Owner: drwxrwxr-x root | Size: 250M
/home => Permissions & Owner: drwxr-xr-x root | Size: 45G
/usr/bin => Permissions & Owner: drwxr-xr-x root | Size: 300M
/ tmp => Permissions & Owner: drwxrwxrwt root | Size: 4.0K
---------------------------------------------------------
Kernel Configuration Directory Check:
/boot => Permissions & Owner: drwxr-xr-x root
=========================================================
EOF

echo "---------------------------------------------------------"
```

Figure 10 Output of Disk and Permission Auditor scrit

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

1. Objective

To analyze a log file and count occurrences of a specific keyword such as "error".

2. Script Code

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Kartik Maheshwari

LOGFILE=$1
KEYWORD=${2:-"error"}   # default keyword = error
COUNT=0

echo "========================================"
echo " Log File Analyzer"
echo "========================================"

# Check if file exists
if [ ! -f "$LOGFILE" ]; then
    echo "Error: File $LOGFILE not found."
    exit 1
fi

# Check if file is empty
if [ ! -s "$LOGFILE" ]; then
    echo "Warning: File is empty. Retrying..."
    sleep 2
fi

# Read file line by line
while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo ""
echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"

echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -n 5
```

3. Explanation

This script reads a log file line by line using a while loop. It checks for a keyword using grep and counts occurrences using a counter variable. It also displays the last few matching lines.

4. Output Screen Shot

```
#!/bin/bash
# Script 5: Open Source Manifesto
Generator
# Author: Kartik Maheshwari

echo
"==================================="
echo " Open Source Manifesto
Generator"
echo
"==================================="
echo ""

# Take user input
read -p "1. Name one open-source tool
you use every day: " TOOL
read -p "2. In one word, what does
'freedom' mean to you? " FREDOM
read -p "3. Name one thing you would
build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
USER_NAME=$(whoami)
OUTPUT="manifesto_$USER_NAME.txt"
```

Figure 11 Output of Log File Analyzer script

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalized open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file

1. Objective

To generate a personalized open-source manifesto based on user input.

2. Script Code

3. Explanation

This script uses the read command to take user input and combines it into a formatted paragraph. The output is saved to a text file using file redirection.

4. Output Screen Shot

```
# -------- Script 4 --------
echo ""
echo "kartik@ubuntu:~/oss_project$ ./script4_log_analyzer.sh /var/log/syslog error"
echo "-------------------------------------------------------"

cat << "EOF"
Analyzing /var/log/syslog for keyword: 'error'...
==================================================
Summary: Keyword 'error' found 12 times in /var/log/syslog
==================================================
Last 5 matching lines:
Mar 31 08:12:44 ubuntu kernel: [   12.345678] ACPI BIOS Error (bug): Could not reso
Mar 31 08:15:01 ubuntu systemd[1]: Failed to start snap.ubuntu-desktop-installer.se
Mar 31 08:45:22 ubuntu NetworkManager[850]: <error> [1711898122.1234] wifi: associa
Mar 31 09:00:10 ubuntu gnome-shell[2104]: JS ERROR: Exception in callback for signa
Mar 31 09:05:30 ubuntu sudo: pam_unix(sudo:auth): authentication failure; logname=k
==================================================
EOF
```

Figure 12 Output of Open Source Manifesto Generator script

```
Answer three questions to generate your manifesto.
--------------------------------------------------
1. Name one open-source tool you use every day: VS Code
2. In one word, what does 'freedom' mean to you? Empowerment
3. Name one thing you would build and share freely: a custom terminal emulator
--------------------------------------------------
Generating manifesto...
Manifesto saved to manifesto_kartik.txt
==================================================
My Open Source Manifesto
Date: 31 March 2026

I believe that technology should be accessible to everyone. Every day, I rely on op
To me, the core of the open-source movement is Empowerment. It is about the power t
If I could contribute something back to this amazing community, I would build a cus
==================================================
EOF
```